DATA EXPLORATION AND VISUALIZATION LAB

Experiment 7 Faculty Manual

Data Visualization Using Matplotlib

Field	Detail
Experiment Title	Create and validate data visualizations using Python and Matplotlib
Course	Data Exploration and Visualization - Python/Power BI
Course Code	25CDSD31
Programme	B.Tech - III Semester
Document Type	Faculty Manual

Designed for: classroom explanation, laboratory execution, record writing, test-case validation, viva preparation and demonstration of correct and incorrect plotting inputs.

Navigation: This PDF contains clickable internal links. Click any item in the Table of Contents to jump to that section.

Complete Faculty Manual • Runnable Python Programs • Test Cases • Output • Viva

Table of Contents

Click any section below to jump directly to it.

1. Aim

To create, customize and validate different types of data visualizations using Python and Matplotlib, including bar charts and histograms, and to safely handle empty datasets and mismatched axis inputs.

2. Objectives

- Understand the purpose of data visualization in data analysis.
- Create categorical comparison charts using a bar chart.
- Create distribution charts using a histogram.
- Add meaningful titles and axis labels.
- Validate plotting input before generating a chart.
- Handle empty datasets without producing misleading visualizations.
- Detect X and Y axis length mismatches before plotting.
- Validate the experiment using success and failure/edge test cases.

3. Expected Learning Outcomes

Outcome	Student will be able to
LO1	Explain the role of data visualization in exploratory analysis.
LO2	Create a bar chart for categorical comparison.
LO3	Create a histogram for numerical distribution.
LO4	Customize charts using titles and axis labels.
LO5	Validate data and safely handle empty or invalid plotting inputs.

4. Prerequisites

- Basic Python syntax, lists and exception handling.
- Basic understanding of categorical and numerical data.
- Python 3.x installed.
- Matplotlib library installed.
- Basic understanding of X-axis, Y-axis, frequency and distribution.

5. Theory

5.1 What is Data Visualization?

Data visualization is the graphical representation of data. Charts and plots convert numerical or categorical values into visual patterns that are easier to compare and interpret. A good visualization communicates the meaning of the data without requiring the reader to inspect every raw value.

5.2 Why Use Matplotlib?

Matplotlib is a widely used Python library for creating static charts and plots. The pyplot module provides simple functions for generating figures, adding labels and displaying visual output.

5.3 Bar Chart

A bar chart compares values across categories. Each category is represented by a separate bar, and the height of the bar represents the corresponding value.

Example Input	Meaning
Department = HR, IT, Finance, Sales	Categories on the X-axis
Employees = 12, 30, 18, 25	Values represented by bar heights

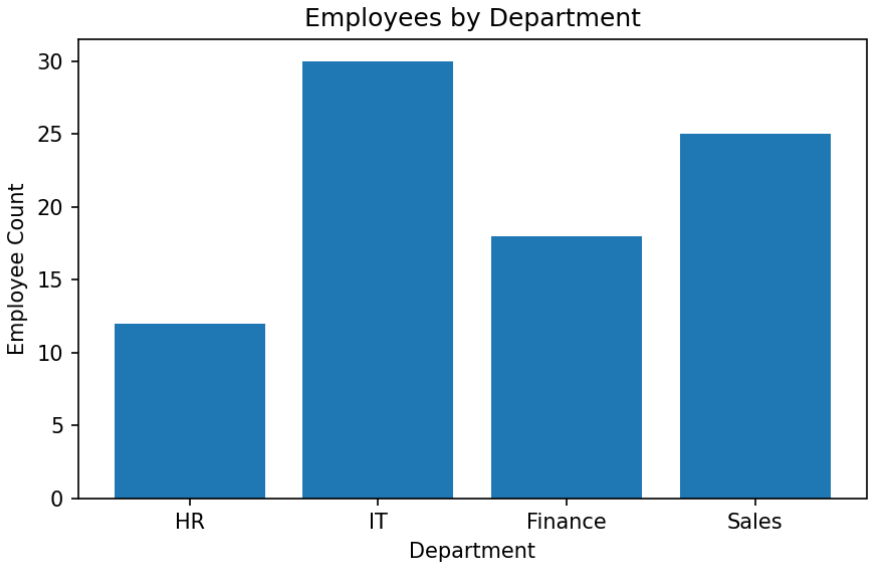

Figure: Employees by Department

5.4 Histogram

A histogram shows the frequency distribution of numerical data. The numeric range is divided into intervals called bins, and the height of each bar represents the number of observations falling within that interval.

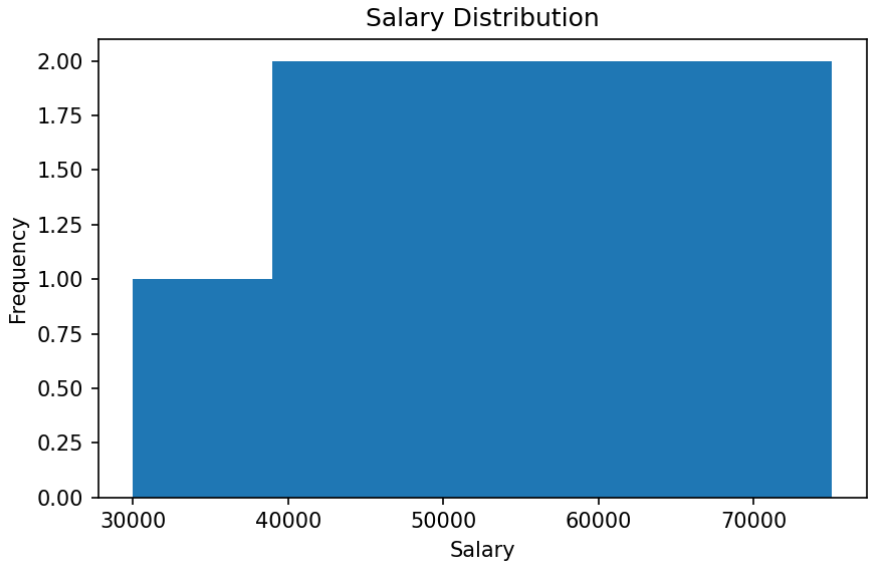

Figure: Salary Distribution

5.5 Difference Between Bar Chart and Histogram

Feature	Bar Chart	Histogram
Data type	Categorical or discrete	Continuous / numerical
Bars	Usually separated	Touch each other because intervals are continuous
Purpose	Compare categories	Show distribution
Example	Employees by department	Salary distribution

5.6 Importance of Labels and Titles

Every visualization should clearly communicate what is being displayed. A title describes the chart, while X-axis and Y-axis labels explain the meaning and units of the plotted values.

6. Assumptions and Data Considerations

- The dataset or list used for plotting should contain valid values.
- A bar chart requires matching category and value information.
- A histogram requires at least one numerical value.
- An empty dataset should be detected before attempting to plot.
- For line or scatter plots, X and Y values must have the same length.
- The number of bins in a histogram affects the apparent distribution.
- Axis labels should describe the data accurately.

7. Software and Libraries Required

Item	Purpose
Python 3.x	Programming language used for visualization.
Matplotlib	Creates charts and plots.
Jupyter / VS Code / PyCharm	Environment for writing and executing code.

Installation Command

```
pip install matplotlib
```

8. Important Functions Used

Function	Explanation
plt.figure(figsize=(w,h))	Creates a plotting area with the required size.
plt.bar()	Creates a bar chart.
plt.hist()	Creates a histogram.
plt.plot()	Creates a line plot.
plt.title()	Adds a chart title.
plt.xlabel() / plt.ylabel()	Adds labels to the axes.
plt.show()	Displays the completed chart.
assert	Validates a required condition before plotting.

9. Variables Used

Variable	Purpose
departments	Stores department names for the bar chart.
employees	Stores employee counts for each department.
salary	Stores numerical salary values for the histogram.
data	Stores input values for testing an empty dataset condition.
x	Stores X-axis values for mismatch testing.
y	Stores Y-axis values for mismatch testing.
e	Stores the exception object when an error is detected.

10. Algorithm

- Start the program.
- Import the Matplotlib pyplot module.
- Prepare categorical department and employee-count data.
- Create a figure and generate a bar chart.
- Add a title and axis labels.
- Display the chart and print the success result.
- Prepare numerical salary data.
- Create a histogram using an appropriate number of bins.
- Validate the salary data and display the histogram.
- Test an empty dataset using an assertion.
- Test mismatched X and Y axis lengths using an assertion.
- Display clear PASS or FAIL messages.
- Stop the program.

10.1 Textual Flowchart

START → Import Matplotlib → Prepare Data → Validate Input → Select Chart Type → Create Figure → Add Title and Labels → Display Chart → Print PASS/FAIL → STOP

11. Complete Python Program with Test Cases

The following code demonstrates successful chart generation and safe handling of invalid input conditions. The same program can be copied directly into a Python file or Jupyter Notebook.

11.1 TEST 1 (Success) – Categorical Data → Bar Chart

```
import matplotlib.pyplot as plt

# Category names and corresponding employee counts
departments = ["HR", "IT", "Finance", "Sales"]
employees = [12, 30, 18, 25]

try:
    # Validate that category and value counts match
    assert len(departments) == len(employees), "Data lengths mismatch"

    plt.figure(figsize=(6,4))
    plt.bar(departments, employees)

    plt.title("Employees by Department")
    plt.xlabel("Department")
    plt.ylabel("Employee Count")

    plt.show()
    print("PASS: Bar Chart Generated")

except Exception as e:
    print("FAIL:", e)
```

Output: PASS: Bar Chart Generated

11.2 TEST 2 (Success) – Distribution Data → Histogram

```
import matplotlib.pyplot as plt

salary = [30000,40000,45000,50000,55000,
          60000,65000,70000,75000]

try:
    # Ensure numerical data exists before plotting
    assert len(salary) > 0, "Dataset is empty"

    plt.figure(figsize=(6,4))
    plt.hist(salary, bins=5)

    plt.title("Salary Distribution")
    plt.xlabel("Salary")
    plt.ylabel("Frequency")

    plt.show()
    print("PASS: Histogram Generated")

except Exception as e:
    print("FAIL:", e)
```

Output: PASS: Histogram Generated

11.3 TEST 3 (Failure/Edge) – Empty Data → No Plot

```
import matplotlib.pyplot as plt

data = []

try:
    # Do not create a plot when the dataset has no values
    assert len(data) > 0, "Dataset is empty"

    plt.figure(figsize=(6,4))
    plt.hist(data)
    plt.show()

except AssertionError as e:
    print("PASS:", e)
```

Output: PASS: Dataset is empty

11.4 TEST 4 (Failure/Edge) – Wrong Axis Input

```
import matplotlib.pyplot as plt

x = [1, 2, 3, 4]
y = [10, 20]

try:
    # X and Y must contain the same number of values
    assert len(x) == len(y), "X and Y axis lengths mismatch"

    plt.plot(x, y)
    plt.show()

except AssertionError as e:
    print("PASS:", e)
```

Output: PASS: X and Y axis lengths mismatch

12. Code Explanation

12.1 Importing Matplotlib

The statement `import matplotlib.pyplot as plt` imports the plotting interface and assigns the short name `plt`.

12.2 Creating a Bar Chart

`plt.bar(departments, employees)` creates a separate bar for each department. The category names are placed on the X-axis and employee counts determine the bar heights.

12.3 Creating a Histogram

`plt.hist(salary, bins=5)` groups salary values into five intervals and shows how frequently values occur within those intervals.

12.4 Validation Using assert

Assertions stop invalid processing early. For example, an empty dataset is rejected before creating a meaningless histogram.

12.5 Exception Handling

The `try-except` structure prevents the program from terminating abruptly and prints a clear PASS or FAIL message for the test case.

13. Program Logic Summary

Stage	Logic
Input	Receive category data or numerical data for visualization.
Validation	Check that required values exist and dimensions are compatible.
Chart Selection	Use bar chart for categories and histogram for distribution.
Customization	Add title and axis labels for clarity.
Visualization	Display the chart using plt.show().
Output	Print a clear success or handled edge-case message.

14. Test Cases

Type	Input	Expected Output	Explanation
Success	Department + employee data	Bar chart generated	Checks categorical comparison
Success	Salary values	Histogram generated	Checks numerical distribution
Failure/Edge	Empty list	Dataset is empty	Prevents meaningless plot
Failure/Edge	X/Y lengths differ	Lengths mismatch	Prevents invalid coordinate plot

14.1 Expected Output Highlights

- Test 1 displays a bar chart comparing employee counts across departments.
- Test 2 displays a histogram showing how salary values are distributed.
- Test 3 safely rejects an empty dataset before any plot is shown.
- Test 4 safely rejects mismatched X and Y axis values.

15. Common Errors and Solutions

Error / Problem	Reason	Solution
ModuleNotFoundError: matplotlib	Library is not installed.	Run: pip install matplotlib
Empty plot	Input list has no values.	Validate using len(data) > 0.
ValueError for X/Y data	Input dimensions are incompatible.	Ensure X and Y have equal lengths.
Misleading histogram	Too few or too many bins.	Choose bins appropriate to the data size and range.
Labels overlap	Figure is too small.	Increase figsize or use tight_layout().
Chart is unclear	Missing title or axis labels.	Add title, xlabel and ylabel.

16. Applications

- Comparing student counts across departments or sections.
- Visualizing sales across product categories.
- Studying salary or marks distributions.
- Comparing website traffic across days or months.
- Monitoring employee, customer or inventory counts.
- Supporting Exploratory Data Analysis before machine-learning tasks.

17. Viva Questions and Answers

Question	Answer
1. What is data visualization?	The graphical representation of data to make patterns and comparisons easier to understand.
2. What is Matplotlib?	A Python library used to create charts and plots.
3. What does pyplot provide?	A simple interface for creating and displaying visualizations.
4. What is a bar chart used for?	Comparing values across categories.
5. What is a histogram used for?	Showing the frequency distribution of numerical data.
6. What is a bin in a histogram?	An interval used to group numerical values.
7. Why add axis labels?	To explain what values each axis represents.
8. What does plt.show() do?	Displays the current figure.
9. Why validate an empty dataset?	To avoid generating a meaningless or misleading chart.
10. Why must X and Y lengths match?	Each X value must correspond to one Y value in coordinate-based plots.
11. What does plt.figure(figsize=(6,4)) do?	Creates a figure with the specified width and height.
12. What is exception handling?	A mechanism for safely responding to runtime errors or invalid conditions.

11.5 TEST 5 (Failure/Edge) – Non-Numeric Data → Invalid Plot Data Detected

This fifth test case validates that numerical data is supplied before creating a histogram. A histogram should not be generated from text values when the experiment expects numeric observations.

```
import matplotlib.pyplot as plt

# Invalid data for a numerical histogram
data = ["HR", "IT", "Finance"]

try:
    # Check that all values are numeric before plotting
    assert all(isinstance(value, (int, float)) for value in data), \
        "Non-numeric data cannot be used for a histogram"

    plt.figure(figsize=(6,4))
    plt.hist(data)
    plt.show()

except AssertionError as e:
    print("PASS:", e)
```

Output: PASS: Non-numeric data cannot be used for a histogram

14.2 Updated Test Case Summary

Type	Input	Expected Output	Explanation
Success	Department + employee data	Bar chart generated	Checks categorical comparison
Success	Salary values	Histogram generated	Checks numerical distribution
Failure/Edge	Empty list	Dataset is empty	Prevents meaningless plot
Failure/Edge	X/Y lengths differ	Lengths mismatch	Prevents invalid coordinate plot
Failure/Edge	Non-numeric histogram data	Invalid data detected	Validates numeric input

18. Faculty Teaching Notes

- Begin by showing students the raw lists before plotting so they understand how data becomes a chart.
- Explain that bar charts compare categories, while histograms show distributions of numerical values.
- Demonstrate title, X-axis label and Y-axis label one by one.
- Run the empty-data test live and explain why no chart should be created.
- Run the X/Y mismatch test and emphasize the importance of input validation.
- Ask students to modify the data and observe how the chart changes.

19. University / Assignment Questions

- Define data visualization and explain its importance in data analysis.
- Differentiate a bar chart and a histogram with examples.
- Write a Python program to generate a bar chart using Matplotlib.
- Write a Python program to generate a histogram using Matplotlib.
- Explain the purpose of titles and axis labels in a visualization.
- Explain why validation is required before generating a plot.
- Develop test cases for empty data and mismatched axis values.

20. Result

Data visualization was performed successfully using Python and Matplotlib. The program generated a bar chart for categorical comparison and a histogram for numerical distribution. Input validation successfully handled an empty dataset and mismatched X/Y axis lengths through clear PASS messages.

21. Conclusion

Matplotlib enables Python programs to convert raw data into meaningful visual representations. Bar charts are useful for comparing categories, while histograms reveal the distribution of numerical values. Proper chart titles, labels and input validation improve both readability and reliability. Testing success and failure/edge conditions ensures that visualizations are generated safely and correctly.